Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/3771678.3771683

RESEARCH-ARTICLE

PhishFind: A Machine Learning-Based System for Real-Time Phishing Detection

DIEGO O. MENDOZA VEGA, University of Lima, Lima, Lima, Peru

DYLAN O. DIAZ MERCADO, University of Lima, Lima, Lima, Peru

EDWIN J. ESCOBEDO CARDENAS, University of Lima, Lima, Lima, Peru

Open Access Support provided by:

University of Lima

PDF Download
3771678.3771683.pdf
31 March 2026
Total Citations: 1
Total Downloads: 1040

Published: 22 November 2025

Citation in BibTeX format

CIIS 2025: 8th International Conference on Systems Engineering - Cybersecurity & AI: Building a reliable digital future
October 1 - 3, 2025
Lima, Peru

CIIS '25: Proceedings of the 2025 8th International Conference on Systems Engineering - Cybersecurity & AI: Building a reliable digital future (October 2025)
https://doi.org/10.1145/3771678.3771683
ISBN: 9798400718809

PhishFind: A Machine Learning-Based System for Real-Time Phishing Detection

Diego O. Mendoza Vega
Carrera de Ingeniería de Sistemas
Universidad de Lima
Lima, Peru
20181183@aloe.ulima.edu.pe

Dylan O. Diaz Mercado
Carrera de Ingeniería de Sistemas
Universidad de Lima
Lima, Peru
20180600@aloe.ulima.edu.pe

Edwin J. Escobedo Cardenas
Carrera de Ingeniería de Sistemas
Universidad de Lima
Lima, Peru
eescobed@ulima.edu.pe

Abstract

Phishing attacks continue to exploit user trust, posing significant risks to individuals and organizations through increasingly sophisticated tactics. Existing detection tools often lack real-time analysis or transparent explanations, leaving a gap in effective browser-based protection. This work introduces `PhishFind`, a browser widget designed to address these limitations by integrating advanced machine learning and explainable AI. As part of this contribution, we developed `PhishingLong`, a continuously updated dataset of phishing and legitimate websites. Leveraging this dataset, the system applies a Gradient Boosting classifier to analyze URLs and web content, while a semantic module based on the ChatGPT API provides users with clear, human-readable explanations for suspicious sites. In evaluation, Gradient Boosting achieved an F1-score of 97.34%, and user testing demonstrated high acceptance, particularly for usability and alert clarity. Overall, `PhishFind` demonstrates the potential of combining robust detection with explainable feedback to enhance user protection against phishing in real time.

CCS Concepts

• **Security and privacy → Artificial immune systems**.

Keywords

Phishing detection, machine learning, malicious domain, cybersecurity

ACM Reference Format:
Diego O. Mendoza Vega, Dylan O. Diaz Mercado, and Edwin J. Escobedo Cardenas. 2025. PhishFind: A Machine Learning-Based System for Real-Time Phishing Detection. In *8th International Conference on Systems Engineering - Cybersecurity & AI: Building a reliable digital future (CIIS 2025), October 01–03, 2025, Lima, Peru.* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3771678.3771683

1 Introduction

Phishing is a social engineering strategy that exploits human vulnerabilities to obtain confidential information, such as login credentials, personal data, or credit card numbers. Attackers use emails, SMS messages, or fraudulent websites to take advantage of users' trust. According to Carroll et al. [12], attackers manipulate victims' perception of trust to push them into rushed decisions. As communication technologies advance and dependence on digital services grows, the opportunities for these threats increase significantly.

As a result, this growing threat affects both individuals and organizations. Attackers trick individuals into providing personal or financial information. These incidents cause identity theft, financial loss, and psychological harm rooted in digital distrust. Companies and governments also face risks, including major economic losses, security breaches, and reputational damage. Aslan et al. [8] report that organizations incur high costs as attacks grow more sophisticated. The adoption of emerging technologies such as the Internet of Things (IoT) expands the attack surface. Furthermore, attackers evade current detection systems and threaten the stability of critical infrastructure.

This trend has accelerated in recent years, with a substantial increase in phishing attacks worldwide. The annual report by ThreatLab [31] indicates that these incidents rose by 47% and impacted sectors previously considered less vulnerable, such as education, which faced a 57.6% increase. The emergence of technologies like artificial intelligence (AI) enables attackers to develop more sophisticated techniques. Kaspersky [18] report that Peru recorded 31.5 million phishing attempts between July 2022 and July 2023, placing it among the most affected countries in Latin America. Furthermore, Kaspersky [19] highlight a 360% growth in phishing activity in Peru compared to the previous year, reaching approximately 145 million attempts and underscoring the accelerated pace of this threat in the region. Cybercriminals manipulate the source code of web pages and apply advanced methods such as "Browser-in-the-Browser" and the smuggling of HTML and SVG files, which complicates detection even further.

The growing scale and complexity of phishing attacks have sparked significant academic interest in detection methods, especially as advances in machine learning and deep learning open new possibilities. Benavides-Astudillo et al. [9] present a model that leverages natural language processing (NLP) and deep learning, achieving over 97% accuracy when analyzing the textual content of web pages. Similarly, Aldakheel et al. [2] implement a CNN-based approach focused on URL features, reaching an accuracy of 98.77%. However, Ejaz et al. [14] warn that machine learning models lose effectiveness over time, so adaptive strategies remain necessary to ensure long-term success. Mohanty et al. [22] introduce a hybrid method that combines URL analysis with web content evaluation, improving accuracy on resource-constrained devices—an essential factor in IoT environments. Despite these advances, most models remain optimized for English-language scenarios, which limits their application in multilingual contexts. Nevertheless, substantial gaps

This work is licensed under a Creative Commons Attribution 4.0 International License.
CIIS 2025, Lima, Peru
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1880-9/25/10
https://doi.org/10.1145/3771678.3771683

persist in phishing detection, particularly in real-time applications and support for additional languages. Most current models focus only on URL analysis and overlook critical aspects of web content, including source code and forms. In addition, many detection systems rely on technologies built exclusively for English-language environments, which restricts their effectiveness in Spanish-speaking contexts.

To bridge these gaps, we introduce `PhishFind`, a browser-integrated widget that uses machine learning and deep learning to analyze both URLs and web content in real time, adapting to various contexts. This tool is supported by `PhishingLong`, a robust dataset that stores and continuously updates information about malicious and legitimate websites. Our goal is to deliver a precise, accessible, and user-friendly phishing detection solution, underpinned by the dynamic capabilities of `PhishingLong`.

2 Related Works

In recent years, research on phishing detection has advanced significantly, driven by the development of machine learning and deep learning models. Researchers have employed diverse techniques that examine both lexical and content-based features of web pages. Nevertheless, conventional datasets present clear limitations, and many authors point out the need for more comprehensive approaches that incorporate the source code of web pages and utilize robust data repositories. Table 1 summarizes the most prominent methods in recent literature, detailing the algorithms and performance metrics relevant to this study.

Several studies demonstrate that effective phishing detection combines lexical analysis of URLs with a deeper inspection of web page content, including the HTML source code. For instance, Rashid et al. [26] use a one-shot learning model based on large language models (LLMs) that analyze lexical features of URLs with chain-of-thought reasoning, achieving high classification accuracy. Both Mohanty et al. [22] and Aljofey et al. [4] stress that URL evaluation alone often fails, as attackers design links that closely mimic legitimate websites using increasingly advanced tactics.

In addition, other researchers have moved beyond URL analysis. For example, Koide et al. [20] propose a system that performs web crawling to extract rendered HTML code, OCR-generated text from screenshots, and URLs. Their approach feeds these data into multimodal models (such as GPT-4V), enabling the detection of social engineering techniques and brand impersonation across multiple languages. This underscores the importance of a comprehensive approach to phishing detection—one that integrates lexical features, web content, and visual analysis to address the evolving tactics of cybercriminals.

Despite advances in analytical techniques, practical implementation still faces significant hurdles. One of the main challenges in phishing detection research lies in maintaining a consistent and up-to-date dataset of phishing sites. Gomes de Barros et al. [16] and Ejaz et al. [14] observe that many phishing pages disappear soon after detection, leaving datasets such as PhishTank outdated or incomplete. This data gap impacts model performance, since models struggle to adapt to emerging threats. To address this issue, Alsharaiah et al. [6] and AS et al. [7] recommend collecting proprietary URLs to train models on current, real-world data and thereby reduce false negatives.

Recent studies also highlight the importance of selecting effective algorithms for phishing detection. Works such as Aljofey et al. [4] and Mohanty et al. [22] confirm the reliability of XGBoost, which adapts well to data variability. Similarly, Alsaedi et al. [5] and Gupta et al. [17] underscore the strengths of Random Forest, including its generalization capability and robustness to data changes. Researchers favor these algorithms because they efficiently process large datasets and identify complex patterns—an essential quality for combating sophisticated phishing attacks. However, results show considerable variability in accuracy and performance metrics depending on the dataset and features in use.

For example, Random Forest and XGBoost achieve accuracy rates near 99.5% on the ISCXURL-2016 dataset, while in newer datasets such as the Mendeley Repository, accuracy decreases to about 97%. This shift occurs because older datasets often contain more standardized URLs that simplify optimization, whereas newer collections include advanced evasion techniques that challenge detection. Abdul Samad et al. [1] note that hyperparameter tuning and careful feature selection can improve accuracy when datasets are well structured. On the other hand, Aljabri et al. [3] and Mohanty et al. [22] argue that less homogeneous datasets like Mendeley require hybrid approaches that combine URL and HTML content analysis. False positive rates also vary—remaining low for structured datasets such as ISCX, but rising in more dynamic ones like Mendeley—which impacts overall precision. This finding agrees with Aljofey et al. [4], who report that recent phishing campaigns use dynamic URL modifications and advanced visual methods that lexical analysis alone cannot detect.

Studies demonstrate that accuracy, F1-score, and false positive rates depend on the quality and diversity of the dataset, along with effective feature selection and algorithm choice. In this context, research by Alsharaiah et al. [6] and Koide et al. [20] confirms that hybrid approaches—integrating lexical, content-based, and contextual analysis—offer the strongest defense against increasingly sophisticated phishing tactics.

Nevertheless, current models still face limitations in adapting to new attack strategies and supporting multiple languages, highlighting the need for innovative solutions that address these gaps.

3 Methodology

This research adopts a methodological framework designed for the development and evaluation of a phishing detection tool. As illustrated in Figure 1, the proposed pipeline covers dataset collection and construction, preprocessing, model training and testing, and integration and deployment. The objective is to detect phishing websites by analyzing both URLs and their source code. The following subsections detail each stage of the methodology, outlining the procedures that ensure the rigor and validity of this work.

3.1 Dataset Construction

The `PhishingLong` dataset construction process involved several stages: link collection, data extraction via web scraping, categorization, and continuous updating (see Figure 2). As the foundation

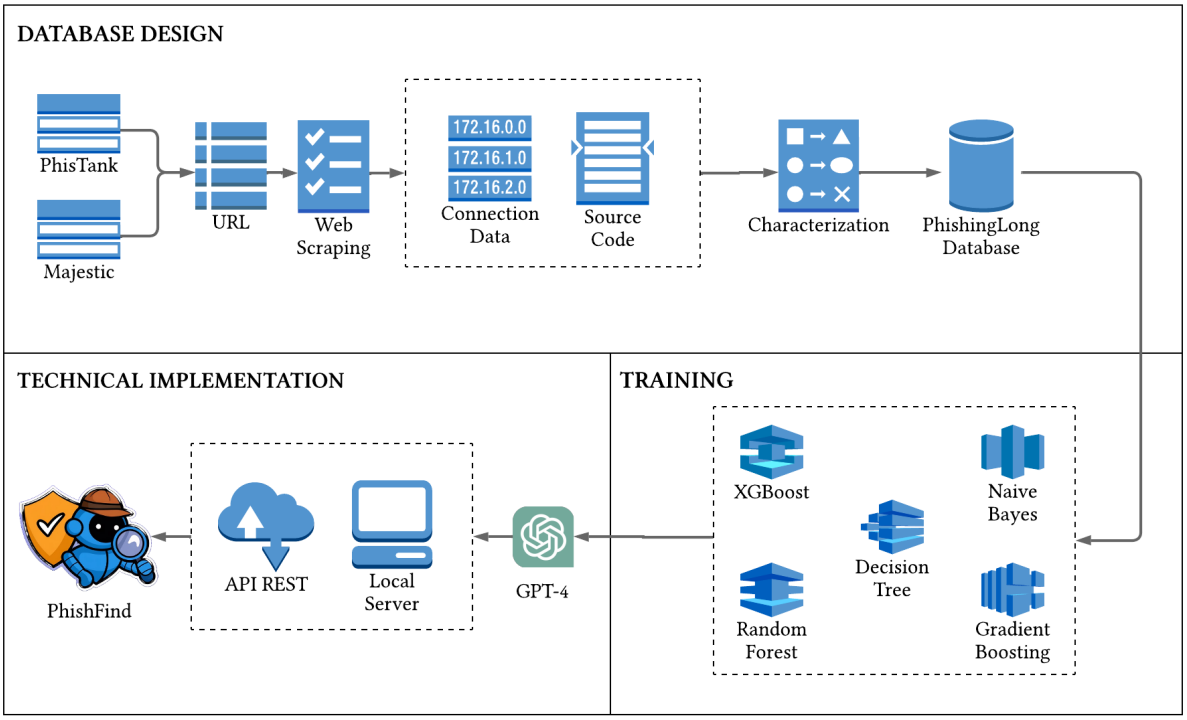


Figure 1: Pipeline of the Proposed Phishing Detection Methodology.

Table 1: Summary of Recent Methods and Performance Metrics for Phishing Detection.

Dataset (with Reference)	Algorithm	Results
PhishTank: 5,000 phishing UNB: 5,000 legitimate [7]	Decision Tree	Accuracy(%) 99.50
PhishTank: 5,000 phishing Common Crawl: 5,000 legitimate [22]	XGBoost	Accuracy: 98.30 Recall: 98.00 Precision: 99.00 F1-Score: 98.00
ISCXURL-2016: 5,982 phishing ISCXURL-2016: 5,982 legitimate [17]	Random Forest	Precision: 99.57 Recall: 99.46 F1-Score: 99.58 AUC: 99.67
OpenPhish, PhishTank: 5,000 phishing Alexa, Common Crawl: 5,000 legitimate [6]	Random Forest + K-Means	Accuracy: 98.64 Recall: 98.70 Precision: 98.60
Mendeley Repository: 5,000 phishing Mendeley Repository: 5,000 legitimate [1]	Gradient Boosting	Accuracy: 98.27
PhishTank: 2,541 phishing Alexa: 2,000 legitimate [24]	Naive Bayes	Accuracy: 99.96
PhishTank: 5,526 phishing Alexa: 5,529 legitimate [21]	MLP	Accuracy: 97.20
PILU-90K: 30,000 phishing PILU-90K: 60,000 legitimate [29]	Logistic Regression	Accuracy: 96.50
ISCX, EBBU, HP datasets (Phishing and benign URLs) [26]	LLM (GPT-4 Turbo, Claude 3)	F1-Score: 92.00
Custom dataset: 1,000 phishing 1,000 non-phishing multilingual websites [20]	Multimodal LLM (GPT-4V, GPT-4)	Precision: 98.7 Recall: 99.6

for data collection, two primary sources in CSV format were used: Phishtank[1], which provides verified phishing links, and Majestic[2], which offers a list of legitimate domains ranked by popularity. From the Majestic dataset, a sample of the top 2,000 domains was selected as a reference for legitimate sites.

Data collection and automation were carried out using libraries such as `widget`, while web crawling enabled the indexing of the corresponding web pages. Due to the short lifespan of phishing websites, critical data such as IP addresses, DNS records, ports, WHOIS queries, and HTML source code were extracted using libraries including `requests` [27], `WHOIS` [32], `BeautifulSoup (BS4)` [28], and `Selenium` [30].

The extracted features were grouped into three categories: URL-based, network-based, and HTML-based. Python functions automated the extraction of these variables, which were then used for training and validating the predictive models. Table 2 presents the variables considered.

Finally, to keep the proposed `PhishingLong` dataset up to date, we implemented a workflow, illustrated in Figure 2, that extracts links from `PhishTank`, checks for their existence in the dataset, and—if they are new—performs preprocessing to generate a JSON file. If data was successfully retrieved, the JSON was added to the dataset; otherwise, the link was recorded as unresponsive. This process prevented duplication and ensured the continuous incorporation of new phishing links. This pipeline operates on a biweekly

[1]https://www.phishtank.com/developer_info.php
[2]https://majestic.com/reports/majestic-million

Table 2: Variables Contained in the Proposed PhishingLong Dataset

Feature	Variable	Explanation
URL	URL	The full address of the website.
	NumDots	Number of dots in the URL, often higher in phishing URLs.
	LenURL	Total length of the URL string.
	NumHyphen	Count of hyphens, which can be used to mimic legitimate domains.
	NumSlash	Number of slashes, indicating URL depth.
	NumArroba	Number of '@' symbols, used to obscure real URLs.
	NumPerc	Count of percent symbols, common in obfuscated URLs.
	NumEqual	Number of equal signs, often in query strings.
	NumTilde	Number of tilde characters in the URL.
	NumUnScore	Number of underscore characters in the URL.
	Deep	The depth of the path in the URL.
	NumAmper	Count of ampersands, used in parameter passing.
	NumDollar	Number of dollar signs, uncommon and potentially suspicious.
	NumNum	Number of numeric digits in the URL.
	LenRoute	Length of the route/path in the URL.
	LenQuery	Length of the query string.
	LenDom	Length of the domain name.
Network	Domain	Registered domain name.
	Subdomain	Subdomain used, which may be manipulated to look trustworthy.
	IP	Whether an IP address is used instead of a domain.
	Port	Port number used in the URL.
	Protocol	Communication protocol (e.g., HTTP, HTTPS).
	Redirection	Presence of redirection in the website.
	Geolocation	Geographic location of the hosting server.
	Cookies	Whether the site uses cookies.
	Download	If automatic downloads are triggered.
	SSL_valid	Whether the SSL certificate is valid.
	SSL_issuer	The certificate authority that issued the SSL certificate.
	SSL_valid_from	Start date of SSL certificate validity.
	SSL_valid_to	End date of SSL certificate validity.
HTML	HTML	Complete HTML content of the page.
	BotNoAct	Non-active bot elements detected.
	BotActInForm	Active bots/scripts detected within forms.
	HypLinAnoDom	Hyperlinks pointing to anonymous or unrelated domains.
	NumForms	Number of form elements on the page.
	NumPopUp	Number of pop-ups, which may be intrusive.
	NumLinkRedi	Number of redirecting links.
	NumImg	Total number of images.
	NumImgRedi	Images loaded from redirecting or external domains.
	NumScript	Count of JavaScript scripts.
	NumVid	Number of embedded videos.
	Favicon	Whether the site uses a favicon.
	FaviExt	Whether the favicon is hosted externally.
	HasIframe	Presence of iframe tags.
	HasFrame	Presence of general frame tags.
	RightClickDisabled	Right-click is disabled via script.
	NoTitle	Whether the page has no title.
	ImgInForm	Images embedded within form tags.
	ExtRedirBot	External redirects triggered by bots/scripts.

schedule to collect the largest possible volume of data from phishing websites. In contrast, the update frequency for legitimate links extracted from Majestic is currently set to annual, since these websites remain significantly more stable compared to phishing sites, which are often taken down within a short period.

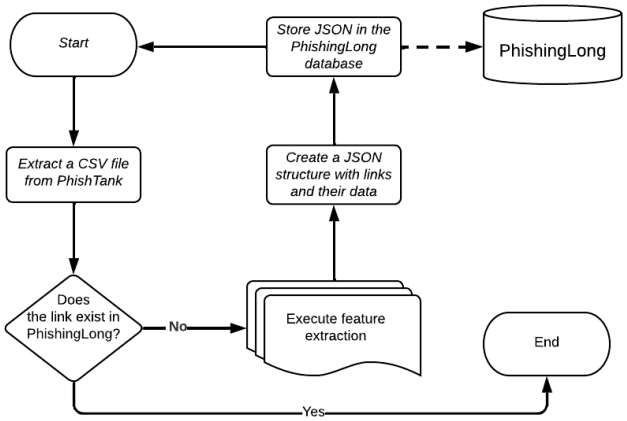


Figure 2: Update Workflow for the Proposed PhishingLong Dataset.

3.2 Model Training and Configuration

The modeling and training process focused on evaluating five classification algorithms for phishing website detection: Gradient Boosting [15], XGBoost [13], Random Forest [10], Decision Tree [25], and Naive Bayes [34]. Each model was configured and trained using the selected features to identify relevant patterns in the dataset. For ensemble methods, such as Gradient Boosting and XGBoost, hyperparameters were tuned to optimize performance on complex and imbalanced data. Random Forest was selected for its stability and resistance to overfitting, while Decision Trees were chosen for their interpretability, and Naive Bayes for its processing speed with categorical features.

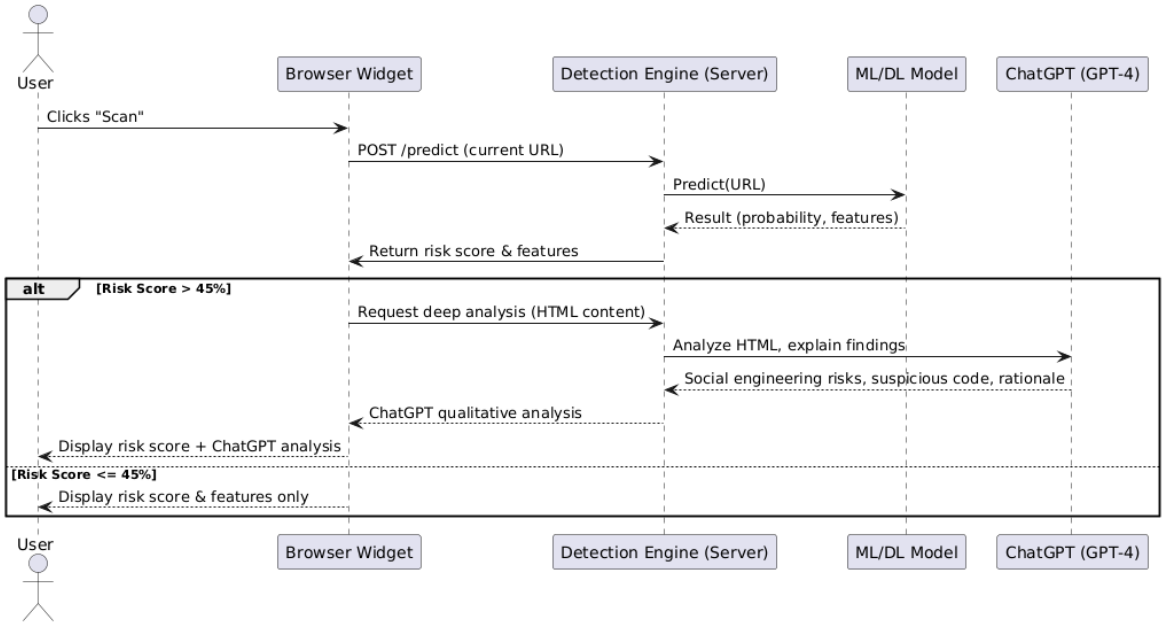
Finally, the `PhishingLong` dataset was split into 70% for training and 30% for testing, with 20% of the training set allocated for validation. Five-fold cross-validation was applied to ensure model robustness. Key evaluation metrics included accuracy, precision, recall, F1-score, and AUC-ROC, all of which are fundamental for assessing cybersecurity models.

3.3 Technical Implementation and Widget Validation

The `PhishFind` detection system consists of two main components: a server-side detection engine and a browser widget. For the purposes of this research, the detection engine ran on a local server that hosted the pre-trained machine learning and deep learning models, together with a REST API for handling user requests via GET and POST methods. This setup enabled real-time predictions and immediate responses to user queries.

The browser widget, developed for Chromium-based browsers, served as the user interface. It featured a "Scan" button that allowed

Figure 3: UML Sequence Diagram of the PhishFind Detection Workflow.



users to analyze the currently active browser tab. The interface displayed the probability that the web page was a phishing site, along with the specific features that contributed to the detection.

The system first uses the machine learning model to evaluate each URL and estimate the probability of phishing based on the extracted features. When a link's risk score exceeds 45% probability of being phishing, the system leverages the ChatGPT (GPT-4) API [23] to provide deeper analysis. In these cases, ChatGPT examines the HTML content, identifies potential social engineering elements, flags suspicious scripts or functions, and explains the rationale behind each finding.

The widget presents this analysis to the user alongside the numerical risk score, combining quantitative and qualitative information to enhance transparency and user trust. The UML sequence diagram in Figure 3 details the step-by-step interaction between the user, widget, detection engine, and prediction model. A practical example appears in Figure 4, which presents the PhishFind interface and demonstrates how the system detects phishing attempts in a specific URL. The figure illustrates how the user can either enter the link directly or obtain it using the Get button, then scan it to visualize the phishing probability percentage. At this stage, a chart briefly displays the features that the model identified as suspicious.

In addition, the widget has a direct interaction with ChatGPT within the workflow. Specifically, when the detection engine assigns a site a risk score greater than or equal to 45% probability of phishing, the system queries the ChatGPT API with a structured prompt. This prompt requests a detailed analysis of the page's HTML code, extracting information such as: number of potentially malicious forms (Form), number of pop-ups (Pop), external images (Img), malicious scripts (ScMl), suspicious scripts (ScSo), third-party videos (Vid), and images with embedded forms (Imc).

The system evaluates the probability social engineering with its justification, identifies problematic code lines, and generates a summary explaining why the page might be considered malicious. The widget then interprets these responses and combines them with the model's numerical score, thus providing both a quantitative measure and a qualitative explanation.

This approach enhances transparency and improves user trust in the tool by offering not only a result but also a clear justification of the detected findings.

4 Results and Discussion

This section presents the results from the training and testing phases of the selected models for phishing website detection. First, we report the performance metrics for each classification algorithm: accuracy, precision, recall, F1-score, and AUC-ROC. Then, we summarize the usability evaluation of the `PhishFind` widget, based on user responses to the Likert scale questionnaire. These results provide a comprehensive assessment of the technical effectiveness and user acceptance of the PhishFind system.

4.1 Models Performance

Table 3 summarizes the performance of the evaluated models on the testing dataset. Gradient Boosting outperformed the other algorithms, achieving an accuracy of 96.71% and an F1-score of 97.34%. This model demonstrated both high precision (97.45%) and recall (97.23%), striking an excellent balance between detecting phishing cases and minimizing false positives and false negatives. These results establish Gradient Boosting as the most reliable choice for critical applications that demand high accuracy.

Furthermore, XGBoost produced competitive results, with an accuracy of 96.60% and an F1-score of 97.25%. Its precision (97.35%)

Figure 4: Example Interface Showing PhishFind's Detection of Phishing and Legitimate Sites.

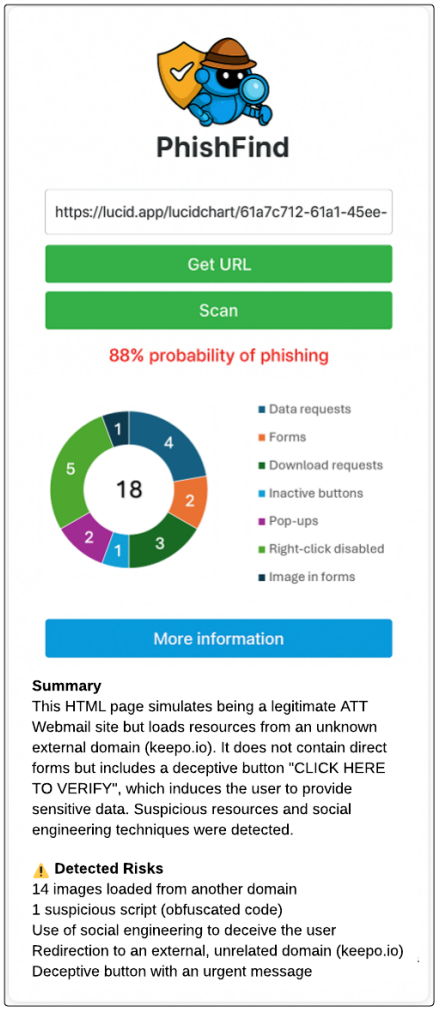


Table 3: Results of the models on the testing dataset.

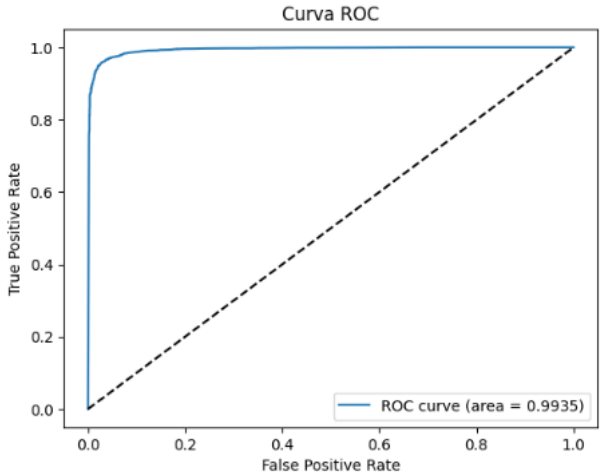
Model	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
Gradient Boosting	96.71	97.45	97.23	97.34
XGBoost	96.60	97.35	97.15	97.25
Random Forest	96.44	97.44	96.79	97.12
Decision Tree	94.72	96.16	95.28	95.72
Naive Bayes	80.03	80.11	90.10	84.81

and recall (97.15%) confirm its suitability for tasks that require both high detection rates and low error margins. In addition, Random Forest achieved an accuracy of 96.44% and an F1-score of 97.12%. While its performance remains slightly below that of Gradient Boosting and XGBoost, this model stands out for its robustness, particularly when stability and resistance to overfitting are valued. Meanwhile, Decision Tree yielded an accuracy of 94.72% and an F1-score of 95.72%. Although less effective than the top-performing models, its simplicity and ease of implementation may benefit use cases with limited computational resources. Finally, Naive Bayes, with an accuracy of 80.03% and an F1-score of 84.81%, ranked lowest among the models tested. Its recall of 90.10% demonstrates reasonable sensitivity, but the lower precision (80.11%) suggests a higher rate of false positives, restricting its application to experimental or low-demand scenarios.

Overall, both Gradient Boosting and XGBoost stand out as the most suitable models for environments requiring high accuracy and reliability in phishing detection. The choice between models should consider specific operational requirements and computational constraints.

The ROC curve for the Gradient Boosting model, shown in Figure 5, further demonstrates its excellent discriminative ability. With an AUC of 0.99, this model achieves near-perfect separation between phishing and legitimate sites, confirming its suitability for the detection task. Based on these results, we selected the Gradient Boosting model as the core detection engine for the proposed `PhishFind` system.

Figure 5: ROC Curve of the Gradient Boosting Model



5 Technical Implementation and Widget Validation

In this section, we evaluate the usability and user acceptance of the `PhishFind` widget following its technical implementation. After deploying the system, twenty-one participants completed a Likert-scale questionnaire that measured different aspects of usability, integration, perceived security, and satisfaction with the tool.

The Likert-scale questionnaire employed in this study was adapted from widely recognized usability assessment instruments such as

Table 4: Likert-scale questionnaire items for PhishFind usability evaluation

Code	Question
Q1	Was it easy to install the widget in the browser you regularly use?
Q2	Did the widget integrate intuitively with active web pages?
Q3	Is the widget's design and interface clear and easy to use?
Q4	Did you receive enough information to understand how the widget works?
Q5	Did the widget correctly identify phishing websites?
Q6	Were the alerts generated by the widget timely and accurate?
Q7	Were there no interruptions or errors while using the widget?
Q8	Did the widget provide useful information on why phishing alerts were triggered?
Q9	Are you satisfied with the overall performance of the widget?
Q10	Would you use this widget regularly to protect yourself from phishing?
Q11	Would you recommend this widget to colleagues, friends, or family?
Q12	Do you think the widget significantly improves your online security?
Q13	Did you feel safer browsing the internet thanks to the widget?

the System Usability Scale (SUS) [11], as well as specific recommendations for evaluating security tools [33]. The selected items address key dimensions, including ease of installation, integration, interface clarity, threat detection accuracy, usefulness of alerts, and overall user satisfaction. This approach provides a comprehensive understanding of user experience and perception when interacting with the anti-phishing widget. The adaptation ensures the validity of the questionnaire for the specific context of the `PhishFind` system, while maintaining comparability with other usability studies.

Table 4 presents the complete set of questionnaire items used to evaluate our proposed approach. Each item asked users to rate their level of agreement on a five-point scale, ranging from "Totally disagree" (1) to "Totally agree" (5). Table 5 presents the responses of all twenty-one participants for each of the thirteen questionnaire items. In addition, Table 6 reports summary statistics, including mean, standard deviation, minimum and maximum scores, and the percentage of agreement (responses of 4 or 5).

Figure 6 displays the distribution of responses for each questionnaire item, grouped by Likert category. The plot shows that the majority of answers concentrate in the "agree" and "totally agree" categories, while lower scores are rare.

Most usability items received mean scores above 4.5, and the percentage of agreement exceeded 85% in nearly all cases. Users rated the overall performance of the widget (Q9) with a mean of 4.71 and 100% agreement, followed by high scores in the clarity of the interface (Q3), the accuracy and timeliness of alerts (Q6), and willingness to recommend the tool (Q11). Even the lowest-scoring item, Q13 ("Did you feel safer browsing the internet thanks to the widget?"), showed a mean of 4.33 and 76% agreement, indicating positive reception and room for further strengthening perceived security.

Finally, the Cronbach's alpha reached 0.90, which indicates excellent internal consistency and reliability in the responses across all items.

These results confirmed that our `PhishFind` system offers a practical solution for browser-based phishing detection. High ratings in installation, interface clarity, and detection accuracy highlight the system's usability and effectiveness. These findings support the possible integration of our approach into broader security strategies.

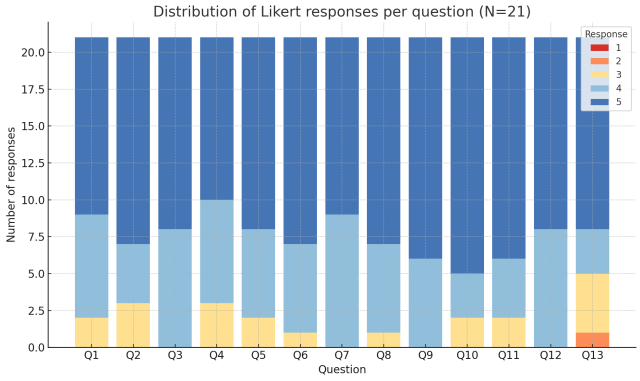
Table 5: Individual User Responses to the Widget Usability Questionnaire (Likert Scale: 5=Totally agree, 1=Totally disagree)

User	Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10	Q11	Q12	Q13
1	5	5	4	4	4	4	4	4	4	4	4	4	3
2	5	5	5	5	5	5	5	5	5	5	5	5	5
3	4	5	5	4	5	4	5	5	5	5	5	5	5
4	4	5	4	5	5	5	4	5	5	5	5	5	5
5	4	3	5	4	3	5	5	5	5	5	5	5	5
6	5	5	5	5	4	5	4	5	4	3	4	4	4
7	4	5	5	3	4	3	5	4	5	5	5	4	3
8	4	3	4	3	4	5	4	3	4	4	3	4	2
9	5	4	4	5	5	4	4	4	4	3	4	4	3
10	3	4	4	4	3	4	4	4	4	5	3	4	3
11	5	5	5	5	5	5	5	5	5	5	5	5	5
12	5	5	5	5	5	5	5	5	5	5	5	5	5
13	5	5	4	3	4	4	4	4	5	5	5	4	4
14	4	3	5	4	4	5	4	4	4	5	5	4	4
15	4	4	4	4	5	5	4	5	5	4	5	5	5
16	5	5	5	5	5	5	5	5	5	5	5	5	5
17	5	5	5	5	5	5	5	5	5	5	5	5	5
18	3	4	4	4	5	4	5	5	5	5	4	5	5
19	5	5	5	5	5	5	5	5	5	5	5	5	5
20	5	5	5	5	5	5	5	5	5	5	5	5	5
21	5	5	5	5	5	5	5	5	5	5	5	5	5

Table 6: Descriptive statistics for Likert questionnaire items (N=21)

Question	Mean	SD	Min	Max	% Agreement (4 or 5)
Q1	4.48	0.68	3	5	90%
Q2	4.52	0.75	3	5	86%
Q3	4.62	0.50	4	5	100%
Q4	4.38	0.74	3	5	86%
Q5	4.52	0.68	3	5	90%
Q6	4.62	0.59	3	5	95%
Q7	4.57	0.51	4	5	100%
Q8	4.62	0.59	3	5	95%
Q9	4.71	0.46	4	5	100%
Q10	4.67	0.66	3	5	90%
Q11	4.62	0.67	3	5	90%
Q12	4.62	0.50	4	5	100%
Q13	4.33	0.97	2	5	76%

Figure 6: Distribution of responses for each questionnaire item, grouped by Likert category (1=Totally disagree, 5=Totally agree).



6 Discussion of Results

The application of Gradient Boosting as the core detection model resulted in consistent and reliable identification of phishing websites, as reflected by an F1-score of 97.34%. This model achieved strong performance using the combined set of lexical and content-based features included in the dataset. XGBoost and Random Forest also achieved robust results, supporting the value of ensemble methods for phishing detection, though Random Forest showed slightly greater susceptibility to false positives. The comparative weakness of models like Naive Bayes (accuracy = 80.03%) illustrates the need for approaches capable of capturing non-linear and complex interactions within phishing data.

In another hand, user testing added another dimension to the proposed system's evaluation. The `PhishFind` widget achieved high usability scores, with most users reporting straightforward installation, intuitive operation, and confidence in the alerts provided. Notably, responses to items about the clarity and relevance of alerts (Q8) indicated that the integration of the ChatGPT API contributed directly to user comprehension: when a suspicious page was flagged, the system supplied concrete reasons—summarizing risky scripts, deceptive prompts, or code fragments—that helped users understand the nature of the threat. This feature addressed the persistent problem of "black box" decision-making in security tools.

Nevertheless, the responses suggest there is still a gap between technical performance and the subjective perception of security, as shown by the lower agreement in Q13 regarding feeling safer online. This gap could be narrowed by further integrating educational components, showing recent blocked threats, or offering short summaries that highlight the widget's activity and protection history. Moreover, the current evaluation is limited in scale and diversity, focusing on a specific user group and language set. Larger deployments and tests with varied user backgrounds and browsing contexts are required to establish generalizability and long-term effectiveness. The reliance on external APIs for ChatGPT analysis also introduces dependency and potential performance variability that should be addressed in future versions.

7 Conclusions

The evidence from both model evaluation and user studies positions PhishFind as an effective tool for phishing detection within web browsers. Gradient Boosting provided stable, high-level detection metrics, while the use of an evolving dataset (our proposed `PhishingLong` dataset) ensured adaptability to recent attack trends. The widget's integration into browsers was straightforward, and users responded positively to its operation and to the explanations generated during phishing alerts.

Improvements remain necessary. Future work will expand multilingual and regional coverage in the dataset, refine the explanation component—potentially replacing the ChatGPT API with lighter or custom-trained models for speed and privacy—and assess real-world performance with larger, more heterogeneous user bases. Incremental or online learning approaches will allow the detection engine to respond to new threats without full retraining. Incorporating hybrid detection mechanisms that analyze visual and behavioral aspects of web pages, alongside textual analysis, is a logical extension. Systematic studies in high-stakes settings, such as corporate or public sector networks, will reveal practical constraints and user needs at scale. Prioritizing clear communication of risk and regular feedback to users will be critical to strengthening both objective security and subjective confidence in the solution.

References

[1] Saleem Raja Abdul Samad, Sundarvadivazhagan Balasubaramanian, Amna Salim Al-Kaabi, Bhisham Sharma, Subrata Chowdhury, Abolfazl Mehbodniya, Julian L Webber, and Ali Bostani. 2023. Analysis of the performance impact of fine-tuned machine learning model for phishing URL detection. *Electronics* 12, 7 (2023), 1642.

[2] Eman Abdullah Aldakheel, Mohammed Zakariah, Ghada Abdalaziz Gashgari, Fahdah A Almarshad, and Abdullah IA Alzahrani. 2023. A deep learning-based innovative technique for phishing detection in modern security with uniform resource locators. *Sensors* 23, 9 (2023), 4403.

[3] Malak Aljabri, Fahd Alhaidari, Rami Mustafa A Mohammad, Samiha Mirza, Dina H Alhamed, Hanan S Altamimi, and Sara Mhd Bachar Chrouf. 2022. An assessment of lexical, network, and content-based features for detecting malicious URLs using machine learning and deep learning models. *Computational Intelligence and Neuroscience* 2022, 1 (2022), 3241216.

[4] A Aljofey et al. 2022. An effective detection approach for phishing websites using URL and HTML features. Sci Rep 12 (1).

[5] Mohammed Alsaedi, Fuad A Ghaleb, Faisal Saeed, Jawad Ahmad, and Mohammed Alasli. 2022. Cyber threat intelligence-based malicious URL detection model using ensemble learning. *Sensors* 22, 9 (2022), 3373.

[6] Mohammad A Alsharaiah, Ahmad Adel Abu-Shareha, Mosleh Abualhaj, Laith H Baniata, Omar Adwan, Adeeb Al-Saaidah, and Majdi Oraiqat. 2023. A new phishing-website detection framework using ensemble classification and clustering. (2023).

[7] Saleem Raja AS, Ganesan Pradeepa, Somasundaram Mahalakshmi, and Manickam S Jayakumar. 2023. Natural language based malicious domain detection using machine learning and deep learning. *Scientific and Technical Journal of Information Technologies, Mechanics and Optics* 23, 2 (2023), 304–312.

[8] Ömer Aslan, Semih Serkant Aktuğ, Merve Ozkan-Okay, Abdullah Asim Yilmaz, and Erdal Akin. 2023. A comprehensive review of cyber security vulnerabilities, threats, attacks, and solutions. *Electronics* 12, 6 (2023), 1333.

[9] Eduardo Benavides-Astudillo, Walter Fuertes, Sandra Sanchez-Gordon, Daniel Nuñez-Agurto, and Germán Rodríguez-Galán. 2023. A phishing-attack-detection model using natural language processing and deep learning. *Applied Sciences* 13, 9 (2023), 5275.

[10] Leo Breiman. 2001. Random Forests. *Machine Learning* 45, 1 (2001), 5–32. doi:10.1023/A:1010933404324

[11] John Brooke. 1996. SUS: A "quick and dirty" usability scale. In *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland (Eds.). Taylor & Francis, London, 189–194.

[12] Fiona Carroll, John Ayooluwa Adejobi, and Reza Montasari. 2022. How good are we at detecting a phishing attack? Investigating the evolving phishing attack email and why it continues to successfully deceive society. *SN Computer science* 3, 2 (2022), 170.

[13] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 785–794. doi:10.1145/2939672.2939785

[14] Asif Ejaz, Adnan Noor Mian, and Sanaullah Manzoor. 2023. Life-long phishing attack detection using continual learning. *Scientific reports* 13, 1 (2023), 11488.

[15] Jerome H Friedman. 2001. Greedy function approximation: a gradient boosting machine. *Annals of Statistics* 29, 5 (2001), 1189–1232. doi:10.1214/aos/1013203451

[16] Julio Cesar Gomes de Barros, Carlo Marcelo Revoredo da Silva, Lucas Candeia Teixeira, Bruno José Torres Fernandes, Joao Fausto Lorenzato de Oliveira, Eduardo Luzeiro Feitosa, Wellington Pinheiro dos Santos, Henrique Ferraz Arcoverde, and Vinicius Cardoso Garcia. 2022. Piracema: a Phishing snapshot database for building dataset features. *Scientific Reports* 12, 1 (2022), 15149.

[17] Brij B Gupta, Krishna Yadav, Imran Razzak, Konstantinos Psannis, Arcangelo Castiglione, and Xiaojun Chang. 2021. A novel approach for phishing URLs detection using lexical based machine learning in a real-time environment. *Computer Communications* 175 (2021), 47–57.

[18] Kaspersky. 2023. *Black Friday 2023: cinco pasos para que empresas protejan su marca contra la suplantación de identidad*. https://latam.kaspersky.com/about/press-releases/black-friday-2023-cinco-pasos-para-que-empresas-protejan-su-marca-contra-la-suplantacion-de-identidad?srsltid=AfmBOopvFxu_eWGEYknWiJiICl3FcXz6KUZ1PbBvkWwxCVFcZyobRJUS&utm_source=chatgpt.com

[19] Kaspersky. 2024. *Aumentan en 140% las estafas mediante mensajes falsos en América Latina, revela Kaspersky*. https://latam.kaspersky.com/about/press-releases/aumentan-en-140-las-estafas-mediante-mensajes-falsos-en-america-latina-revela-kaspersky?srsltid=AfmBOooVJspQAo8XBqcyM0o7m6UxDs84RD1FAvCSyzKOnlG9Z9yxnipw

[20] Takashi Koide, Naoki Fukushi, Hiroki Nakano, and Daiki Chiba. 2023. Detecting phishing sites using chatgpt. *arXiv preprint arXiv:2306.05816* (2023).

[21] C Vineeth Krishna, C Narayana Swamy, A Viji Amutha Mary, and Mercy Paul Selvan. 2021. Identification of Phishing Urls Using Machine Learning. In *Journal of Physics: Conference Series*, Vol. 1770. IOP Publishing, 012009.

[22] Sanjukta Mohanty, Arup Abhinna Acharya, Tarek Gaber, Namita Panda, Esraa Eldesouky, and Ibrahim A Hameed. 2024. An efficient hybrid feature selection technique towards prediction of suspicious URLs in IoT environment. *IEEE Access* (2024).

[23] OpenAI. 2023. GPT-4 Technical Report. https://platform.openai.com/docs/models/gpt-4. Accessed: 2025-07-02.

[24] Abdul A Orunsolu, Adesina S Sodiya, and Adio Taofeek Akinwale. 2022. A predictive model for phishing detection. *Journal of King Saud University-Computer and Information Sciences* 34, 2 (2022), 232–247.

[25] J. Ross Quinlan. 1986. Induction of decision trees. *Machine Learning* 1, 1 (1986), 81–106. doi:10.1007/BF00116251

[26] Fariza Rashid, Nishavi Ranaweera, Ben Doyle, and Suranga Seneviratne. 2025. LLMs are one-shot URL classifiers and explainers. *Computer Networks* 258 (2025), 111004.

[27] Kenneth Reitz and contributors. 2024. Requests: HTTP for Humans. https://pypi.org/project/requests/. Accessed: 2024-07-01.

[28] Leonard Richardson. 2024. Beautiful Soup Documentation. https://www.crummy.com/software/BeautifulSoup/bs4/doc/. Accessed: 2024-07-01.

[29] Manuel Sánchez-Paniagua, Eduardo Fidalgo Fernández, Enrique Alegre, Wesam Al-Nabki, and Víctor González-Castro. 2022. Phishing URL detection: A real-case scenario through login URLs. *IEEe Access* 10 (2022), 42949–42960.

[30] SeleniumHQ and contributors. 2024. Selenium Documentation. https://www.selenium.dev/documentation/. Accessed: 2024-07-01.

[31] Zscaler ThreatLab. 2023. Zscaler ThreatLabz 2023 Phishing Report.

[32] Markus Unterwaditzer and contributors. 2024. python-whois. https://pypi.org/project/python-whois/. Accessed: 2024-07-01.

[33] Melanie Volkamer and Karen Renaud. 2013. Mental models–general introduction and review of their application to human–centred security. In *Human Aspects of Information Security, Privacy, and Trust*, Theo Tryfonas and Ioannis Askoxylakis (Eds.). Springer, 255–265.

[34] Harry Zhang. 2004. The Optimality of Naive Bayes. In *Proceedings of the 17th International Florida Artificial Intelligence Research Society Conference (FLAIRS 2004)*. 562–567. https://dl.acm.org/doi/10.5555/2074158.2074196